

Título do Trabalho:

Desafio de Automação Digital: Gestão de Peças, Qualidade e Armazenamento

Aluno:

Marcio Orlandini

1. Contextualização do Desafio: A Importância da Automação na Indústria

O desafio apresentado simula um cenário comum na indústria moderna: a necessidade de migrar de processos manuais para sistemas automatizados. Atualmente, a empresa fictícia utiliza uma inspeção manual para controle de produção e qualidade, um método que gera atrasos, falhas de conferência e aumento no custo de operação.

A automação, neste contexto, é crucial por diversas razões:

- **Precisão e Consistência:** Diferente de um inspetor humano, que pode sofrer de cansaço ou distração, um sistema automatizado aplica os critérios de qualidade (peso, cor, comprimento) de forma idêntica e rigorosa em 100% das peças, 24 horas por dia.
- **Redução de Custos:** Embora exija um investimento inicial, a automação diminui os custos operacionais a longo prazo, reduzindo o desperdício de matéria-prima (peças reprovadas incorretamente) e os custos de mão de obra associados a tarefas repetitivas.
- **Geração de Dados (Data-Driven):** A automação não apenas executa uma tarefa, mas também coleta dados. Como visto em nosso protótipo, o sistema gera relatórios consolidados sobre o total de peças aprovadas, reprovadas e, mais importante, os motivos da reprovação. Isso permite que a gestão identifique falhas na linha de produção (ex: "a máquina X está descalibrada, pois muitas peças estão falhando no peso") e tome decisões estratégicas.

2. Estrutura do Raciocínio Lógico

Para construir um sistema funcional que atendesse a todos os requisitos, a lógica de programação foi estruturada da seguinte maneira:

- **Funções (Modularização):** O código foi dividido em funções específicas para cada responsabilidade. Criamos `cadastrar_nova_pecas()`, `gerar_relatorio_final()`, `armazenar_em_caixa()`, etc. Isso torna o código mais limpo, fácil de ler, reutilizável e simples de dar manutenção.
- **Decisões (Condições):** A lógica de "Aprovado vs. Reprovado" é o núcleo do sistema. Utilizamos estruturas `if`, `else` e `not` para avaliar cada critério de qualidade. Por exemplo: `if not (95 <= peso <= 105):` verifica se a peça está *fora* do padrão de peso aceitável.
- **Repetição (Loops):** Foi utilizado um loop `while True` para criar o menu interativo. Esse loop garante que o programa continue executando, permitindo ao usuário cadastrar várias peças ou consultar relatórios, até que ele decida ativamente sair (digitando a opção "0").

- **Armazenamento (Listas):** Para simular um "banco de dados" simples, utilizamos listas do Python. `pecas_aprovadas` e `pecas_reprovadas` guardam o histórico. O desafio do armazenamento em caixas foi resolvido com uma "lista de listas" (`caixas = [[]]`), onde a última lista do conjunto é sempre a "caixa aberta" atual.

3. Benefícios da Solução e Desafios do Desenvolvimento

- **Benefícios Percebidos:**
 - **Rastreabilidade Total:** O sistema fornece um ID único para cada peça, permitindo rastrear seu status (aprovada/reprovada) e sua localização (em qual caixa está).
 - **Gestão Logística Otimizada:** A lógica de fechar caixas automaticamente com 10 peças já prepara o lote para armazenamento ou transporte, otimizando o fluxo de trabalho.
 - **Dashboard de Falhas:** O relatório de motivos de reprovação é a ferramenta de gestão mais valiosa do protótipo, transformando dados brutos em inteligência acionável.
- **Desafios Enfrentados:**
 - **Gestão das Caixas:** O desafio lógico mais complexo foi o armazenamento. Garantir que a peça fosse para a caixa correta e que uma nova caixa fosse iniciada (`caixas.append([])`) exatamente quando a anterior atingisse 10 peças exigiu um controle cuidadoso da "lista de listas".
 - **Remoção de Peças:** Implementar a função "Remover peça" foi um desafio adicional. O sistema precisava não apenas remover a peça da lista `pecas_aprovadas`, mas também encontrá-la e removê-la da caixa específica onde havia sido guardada.

4. Reflexão Final: Expansão para um Cenário Real

Este protótipo em Python é o "cérebro" do sistema de automação. Para expandi-lo de um protótipo para uma solução industrial real ("Indústria 4.0"), poderíamos:

- **Integração com Sensores (Hardware):** Substituiríamos os comandos de `input()` por integrações reais. O script receberia dados diretamente de:
 - Uma **balança digital** na esteira (para o peso).
 - Um sistema de **Visão Computacional com IA** (uma câmera) para analisar a cor e medir o comprimento em milissegundos.
- **Integração Industrial (Software):** O sistema deixaria de usar listas e passaria a salvar os dados em um banco de dados robusto (como SQL ou MongoDB). Ele também se comunicaria com outros sistemas da fábrica, como o ERP (Sistema de Gestão Empresarial), para informar o estoque de peças aprovadas em tempo real.
- **Ação Física (Robótica):** Ao invés de apenas `print("Peça REPROVADA")`, o script Python enviaria um comando (via CLP - Controlador Lógico Programável) para um braço robótico ou um pistão pneumático, que fisicamente descartaria a peça defeituosa da linha de montagem.